behaviour. They share a starting point, but their final shapes are different.

Begin with the job, not the dataset

Before downloading a single document, define one task. Imagine a private assistant answering questions about an electronics retailer's product policies. It needs current warranty terms, returns procedures, product specifications and escalation rules. A dump of promotional posts and every customer conversation ever recorded would introduce noise and potentially sensitive data.



Write a one-page data specification containing five fields: intended user, questions the assistant must answer, authoritative sources, update frequency and forbidden information. Then collect ten realistic questions, such as "Can an opened laptop be returned after delivery?" and "What proof is required for warranty service?" Their answers will guide what you collect and later help you test retrieval.

Separate facts from examples. Policies and catalogues are knowledge: the application should look them up when needed. A collection of correctly formatted support replies is training material: it illustrates how the assistant should respond. Do not confuse the two. Retraining a model whenever a return window changes is avoidable work; retrieving a revised policy is much more sensible.

Choose an initial target you can inspect manually: perhaps twenty approved documents and thirty examples. Quality is more useful than volume at this stage.

Collect without losing the source

Return to the `my-ai` workspace from Chapter 1. Inside `data`, create `raw`, `processed`, `training` and `evaluation` folders. Treat `raw` as read-only. Never overwrite the originals when cleaning material; you may discover that an extraction script silently dropped a table or heading.

Step 1: Inventory your files. Make a spreadsheet with columns for file name, source URL or owner, document type, publication date, version, permission status and intended use. For every item, record whether it is an official source, a working draft or an obsolete copy. Exclude material you cannot legitimately use.

Step 2: Extract common formats. Plain TXT and Markdown are straightforward because their contents are already text. For a text-based PDF, install PyMuPDF with `pip install pymupdf`, then open the file using `pymupdf.open("data/raw/policy.pdf")`. Iterate through pages and call `page.get_text()`, saving the results with page numbers. PyMuPDF warns that text order in some PDFs may not match visual reading order; inspect columns and footnotes carefully.

For DOCX files, `python-docx` can read paragraphs and tables. Do not discard tables: a two-column "problem/solution" chart may contain the most valuable information in the document. For CSV files, use `pandas.read_csv()` and check column names, missing values and inferred types. Read identifiers such as product codes as text so leading zeros are not lost.

For database exports, select only the fields needed by the task, and retain a reliable record identifier. Chat histories require extra care: separate speakers, timestamps and quoted replies, or a model may mistake an old customer complaint for an official policy. FAQs should preserve each question-answer pairing instead of joining every answer into one continuous document.



Step 3: Handle websites selectively. Save approved pages with their URLs and retrieval dates. Trafilatura can extract main article content without much of the surrounding navigation: `fetch_url(url)` downloads a page and `extract(html)` retrieves the text. Some sites require JavaScript, and extraction can fail. Respect access restrictions, terms and robots directives; never assume a publicly viewable page is licensed for training.